GUÍA INTEGRAL DE TRABAJO

Prueba Corta 1 · Base de Datos Avanzadas · Caso 9: Gimnasio

Integrantes: Iván Barboza Blanco y Wilberth
Universidad: Universidad Técnica Nacional (UTN)
Repositorio: github.com/IvanB012/PruebaCorta1-Gimnasio
Fecha de referencia: octubre de 2026

Propósito del documento

Esta guía funciona como plan de coordinación, manual operativo y contexto compartible con asistentes de IA. Explica qué hacer, por qué, en qué orden, qué entregar y qué evidencias conservar. No reemplaza el enunciado oficial ni pretende presentar pruebas aún no ejecutadas.

Estado confirmado al preparar esta guía

• Dataset generado con los dos carnés y el caso correcto.

• Contenedores Docker con nombres correctos verificados por Iván; PostgreSQL iniciado.

• PostgreSQL cargado: 6.363 afiliados y 317.007 pagos; conteos confirmados mediante SQL.

• Repositorio clonado y push inicial exitoso (commit fd86f65).

• Ramas main, ivan-postgresql y wilberth-nosql publicadas en remoto.

Pendiente de verificar: carga de MongoDB y Redis, evidencias de consultas A2-A4/B2-B4, comparación de índices, informe final y video.

Identificador	Valor
Carnés	208820443 y 208830343
Parámetros	PAIS = CO · EDAD = 50 · ANIO = 2024 · MONTO = 150
Redis	USUARIO = u241 · sesion:u241
Integridad	316164E0

1. Qué pide el proyecto y qué se entrega

La prueba se realiza en parejas e incluye actividades en MongoDB, Redis y PostgreSQL, comparación de motores, documentación y demostración audiovisual. La entrega consta de informe Word/PDF con comandos, resultados y capturas, y video de máximo 10 minutos donde participan ambos. La fecha de entrega debe confirmarse en el aula: el enunciado la sitúa en la semana 6.

Bloque	Apartados	Puntos
A. MongoDB/Redis	A1 filtro; A2 agregación; A3 explain e índice; A4 sesión, contador y TTL	25
B. PostgreSQL	B1 conteos; B2 JOIN y EXPLAIN inicial; B3 índice/optimización; B4 antipatrones	32
C. Integración escrita	C1 equivalencias; C2 reflexión técnica; C3 forma y evidencia	21
Video	Demostración conjunta, consultas en 3 motores y comparaciones de explain	22
Total		100

Reglas de evaluación que no deben perderse

• A3 y B2/B3 requieren planes antes y después del índice. No construir los índices hasta guardar el estado inicial.

• El video debe incluir ambos integrantes hablando, pantalla compartida, ejecución real en los tres motores y EXPLAIN antes/después.

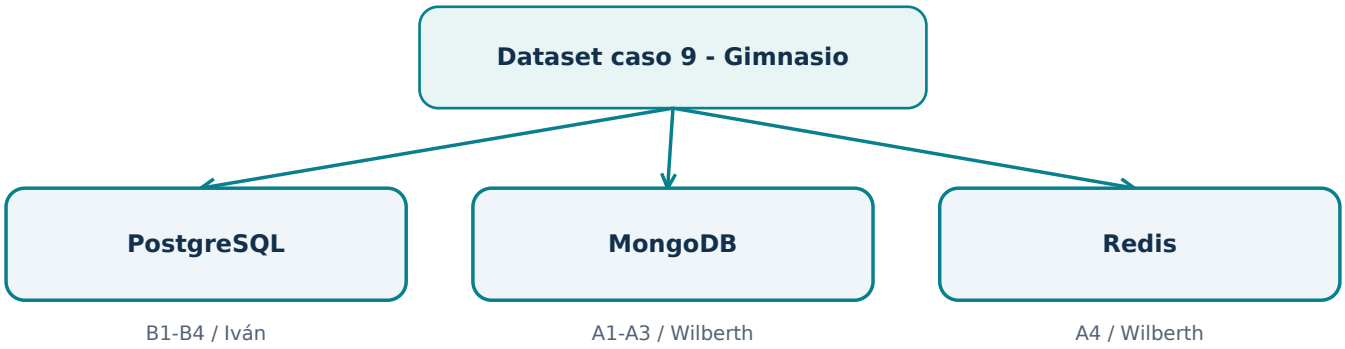
• El informe debe mostrar portada con carnés, caso, ficha e integridad, comandos legibles y evidencia trazable.

• La actividad aplica penalización por entrega tardía según el documento de la prueba.

Vocabulario personalizado que reemplaza ejemplos genéricos

Genérico del enunciado	Caso Gimnasio
clientes	afiliados
pedidos	pagos
cliente_id	afiliado_id
PAGADO / PENDIENTE / CANCELADO	ACREDITADO / PENDIENTE / RECHAZADO

2. Modelo de datos y arquitectura



No hay integración técnica entre motores; la integración es informe + video.

La misma generación determinista produce datos para las tres tecnologías. No se desarrolla una API ni se sincronizan las bases entre computadoras: cada integrante trabaja contra sus contenedores locales. El documento y las evidencias son los artefactos de integración.

Estructura verificada de PostgreSQL

```
afiliados(id PK, nombre, pais, edad, activo)
pagos(id PK, afiliado_id FK, fecha, monto, estado)
```

Relación: cada pago pertenece a un afiliado por medio de pagos.afiliado_id = afiliados.id. En MongoDB existe la colección afiliados. En Redis se gestionan claves de sesión y un contador de visitas.

3. Distribución de responsabilidades

Iván — Parte B	Wilberth — Parte A
B1. Contar afiliados y pagos.	A1. Contar afiliados con pais CO, edad > 50, activo true.
B2. JOIN: cinco afiliados de CO con mayor total ACREDITADO en 2024; plan inicial.	A2. Agrupar por pais, contar y ordenar cantidad descendente, pais ascendente.
B3. Evaluar índices, ejecutar ANALYZE y comparar planes, operadores y tiempos.	A3. executionStats antes/después del índice compuesto pais+edad.
B4. Corregir SELECT * y EXTRACT(YEAR FROM fecha), demostrar equivalencia y rendimiento.	A4. Leer sesion:u241, contador +5, TTL 120s y explicación sobre Redis.

Trabajo compartido: C1 tabla de equivalencias SQL/Mongo/Redis; C2 reflexión de 8-12 líneas; C3 armado final; video, revisión y envío.

Distribución de la documentación

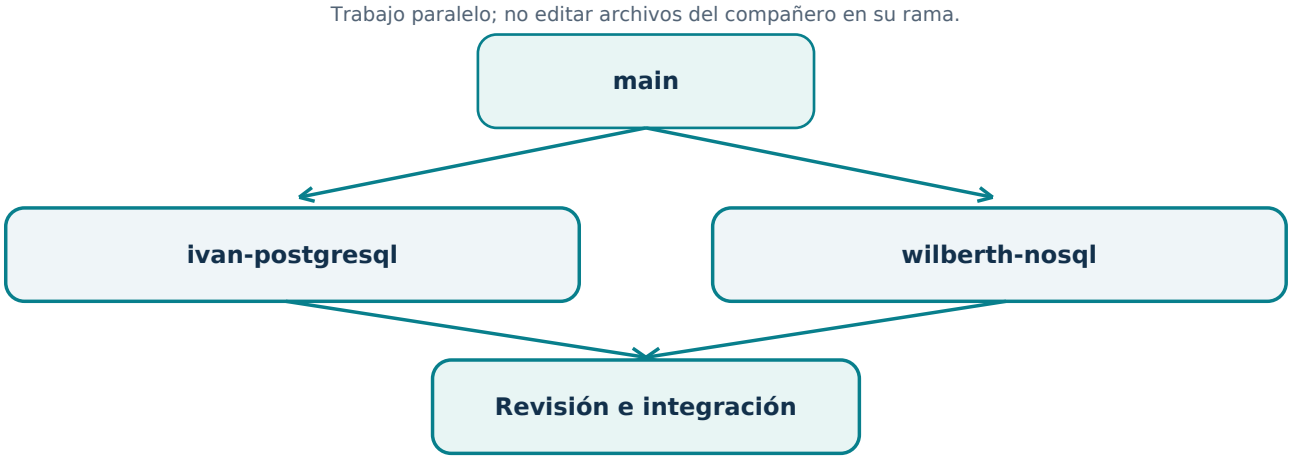
• Cada uno escribe una explicación sencilla de su consulta, el resultado observado y la captura correspondiente.

• Cada uno conserva sus archivos ejecutables y evidencias bajo su carpeta personal.

• Las mediciones de tiempo son locales: no copiar cifras del otro equipo ni inventar porcentajes.

• Ambos revisan el informe completo, la integridad del caso y la duración del video antes de entregar.

4. Flujo de GitHub y coordinación



Regla: cada uno desarrolla en su rama. La rama main representa la base revisada. Wilberth deberá aceptar la invitación a GitHub para poder hacer push. La rama ya fue creada por Iván, por lo que Wilberth debe cambiar a ella, no crear otra con el mismo nombre.

Comandos iniciales de Wilberth (PowerShell)

```
git clone https://github.com/IvanB012/PruebaCorta1-Gimnasio.git
cd PruebaCorta1-Gimnasio
git switch wilberth-nosql
git status
```

Ciclo individual para cada integrante

```
git status
git add consultas/ evidencias/
git commit -m "feat: avances de consultas y evidencias"
git push
```

Antes de hacer merge, cada uno revisa su propia rama en GitHub; abran Pull Requests contra main, revisen archivos/capturas y fusionen uno por vez. Si un archivo cambia en ambas ramas, resuelvan el conflicto juntos. Eviten editar simultáneamente el mismo informe final.

5. Estructura recomendada del repositorio

```
PruebaCorta1-Gimnasio/
  README.md                 .gitignore
  generar_dataset.py
  datos/                    ficha.txt  postgres.sql  afiliados.json
                            redis.txt  cargar.txt
  consultas/                postgresql/  mongodb/  redis/
  evidencias/               ivan/postgresql/
                            wilberth/mongodb/  wilberth/redis/
  informe/                  Informe_PruebaCorta1.pdf (al finalizar)
  video/                    guion.md  enlace_video.txt (si aplica)
```

Mantener los archivos generados del profesor sin alterarlos. No subir contraseñas nuevas, variables de entorno ni información personal no necesaria. La ficha contiene carnés: es recomendable conservar el repositorio privado.

6. Preparación y carga de los motores

6.1 Verificar el entorno

```
docker ps -a
docker start isw623_db isw623_mongo isw623_redis
docker ps
```

Reutilicen los contenedores de clase. Las bases de datos que se crean o cargan son locales; Git no publica la información ya insertada en Docker. Por ello, Wilberth debe hacer su propia carga local.

6.2 PostgreSQL — Iván (ya realizado)

En PowerShell, desde la carpeta datos (contiene postgres.sql):

```
Get-Content postgres.sql | docker exec -i isw623_db psql -v ON_ERROR_STOP=1 -U estudiante -d laboratorio1
```

Este SQL elimina y recrea las tablas afiliados y pagos. No repetirlo si ya se cargó y se conservaron pruebas o índices. La carga de Iván mostró COPY 6363, COPY 317007 y dos ANALYZE.

```
docker exec isw623_db psql -U estudiante -d laboratorio1 -c "SELECT (SELECT COUNT(*) FROM afiliados) AS
total_afiliados, (SELECT COUNT(*) FROM pagos) AS total_pagos;"
```

Resultado verificado por Iván: 6363 y 317007. La captura B1_conteo_inicial_postgresql.png debe mostrar comando y salida.

6.3 MongoDB — Wilberth (pendiente)

Ejecutar desde la carpeta datos que contiene afiliados.json. El comando del profesor copia el JSON al contenedor y luego lo importa:

```
docker cp afiliados.json isw623_mongo:/tmp/afiliados.json
docker exec isw623_mongo mongoimport -u estudiante -p isw623 --authenticationDatabase admin --db pc1
--collection afiliados --drop --jsonArray --file /tmp/afiliados.json
```

La opción --drop borra previamente la colección afiliados de pc1. Wilberth verificará el conteo y capturará la salida si es relevante, antes de los apartados A1-A3.

6.4 Redis — Wilberth (pendiente)

El archivo redis.txt empieza por FLUSHALL: borra todas las claves del Redis seleccionado. No ejecutarlo si hay datos de otras prácticas que deban conservarse.

```
Get-Content redis.txt | docker exec -i isw623_redis redis-cli
```

Al cargar, se crean 300 sesiones y un contador inicial. La sesión seleccionada sesion:u241 almacena un JSON con pais PA y carrito 1. Ese PA es dato de sesión, distinto del parámetro general PAIS=CO.

7. Guía de Wilberth — MongoDB y Redis (A1–A4)

A1 · Filtro compuesto, 6 puntos

Objetivo: contar afiliados del país CO, mayores de 50 años y activos. Usar la colección afiliados de pc1. Ejemplo de consulta para revisar y ejecutar:

```
db.afiliados.countDocuments({ pais: "CO", edad: { $gt: 50 }, activo: true })
```

Guardar consulta, cantidad real y captura A1_filtro_mongodb.png. No escribir un resultado anticipado sin ejecutar.

A2 · Agrupación por país, 4 puntos

Agrupar afiliados por pais, contar los documentos y ordenar la cantidad de mayor a menor, luego el país alfabéticamente en caso de empate.

```
db.afiliados.aggregate([
  { $group: { _id: "$pais", cantidad: { $sum: 1 } } },
  { $sort: { cantidad: -1, _id: 1 } }
])
```

Guardar A2_agrupacion_paises_mongodb.png. Describir qué hace $group y por qué hay un orden secundario por país.

A3 · explain e índice, 8 puntos

Secuencia estricta: (1) comprobar índices existentes; (2) ejecutar filtro con explain("executionStats") antes; (3) guardar captura y valores stage, totalDocsExamined y nReturned; (4) crear índice {pais:1, edad:1}; (5) repetir explain; (6) comparar. Mantener mismo filtro en ambos planes.

```
db.afiliados.getIndexes()
db.afiliados.find({ pais: "CO", edad: { $gt: 50 }, activo: true }).explain("executionStats")
// Solo después de guardar el plan inicial:
db.afiliados.createIndex({ pais: 1, edad: 1 })
```

El campo stage puede estar anidado en winningPlan y puede haber FETCH, IXSCAN o COLLSCAN. No afirmar que el índice aceleró el caso sin contrastar resultados. Capturas A3_antes_indice.png y A3_despues_indice.png.

A4 · Sesión, contador y TTL, 7 puntos

Desde redis-cli, leer la sesión seleccionada; interpretar pais y carrito del JSON. Registrar valor inicial del contador, aplicar INCR cinco veces (cinco comandos o un ciclo externo), comprobar valor final, aplicar TTL de 120 segundos y capturar inmediatamente el TTL positivo.

```
GET sesion:u241
GET contador_visitas
INCR contador_visitas
INCR contador_visitas
INCR contador_visitas
INCR contador_visitas
INCR contador_visitas
EXPIRE sesion:u241 120
TTL sesion:u241
```

Si nadie ha modificado el contador tras la carga, inicia en 8298 y cinco incrementos llegan a 8303; verificarlo en el contenedor. TTL disminuye con el tiempo; no esperar para tomar A4_ttl_redis.png. Explicar que Redis no utiliza el mismo EXPLAIN de planes SQL/Mongo para estas operaciones de clave.

8. Guía de Iván — PostgreSQL (B1–B4)

B1 · Conteos, 4 puntos (ya verificados)

```
SELECT COUNT(*) FROM afiliados;
SELECT COUNT(*) FROM pagos;
```

Valores comprobados: 6363 y 317007. Evidencia: B1_conteo_inicial_postgresql.png.

B2 · JOIN y plan inicial, 8 puntos

Problema del caso: determinar los cinco afiliados de Colombia con mayor suma de pagos ACREDITADO en 2024. Se necesita JOIN entre afiliados y pagos, filtro por país/estado/fecha, SUM(monto), GROUP BY, orden descendente y LIMIT 5.

Consulta modelo inicial, sin crear aún índices adicionales:

```
SELECT a.id, a.nombre, SUM(p.monto) AS monto_total
FROM afiliados a JOIN pagos p ON p.afiliado_id = a.id
WHERE a.pais = 'CO'
  AND p.estado = 'ACREDITADO'
  AND p.fecha >= DATE '2024-01-01'
  AND p.fecha < DATE '2025-01-01'
GROUP BY a.id, a.nombre
ORDER BY monto_total DESC, a.id
LIMIT 5;
```

```
EXPLAIN (ANALYZE, BUFFERS) <consulta B2 sin punto y coma final>;
```

Ejecutar la consulta y el EXPLAIN real; documentar costo estimado, tiempo, rows, buffers y operadores clave. Se puede mantener la consulta base en un archivo B2.sql y el resultado en una captura. Guardar B2_explain_antes_indice.png antes de tocar los índices.

B3 · Índices y rendimiento, 12 puntos

Diseñar índices en función de filtros y JOIN; justificar su orden y costo en escrituras. No asumir que una combinación específica siempre mejora el rendimiento: el optimizador puede optar por Hash Join, Seq Scan o Index Scan según volumen y selectividad. Tras crear índices, ejecutar ANALYZE y repetir EXPLAIN (ANALYZE, BUFFERS) de la misma consulta.

```
-- Ejemplos para evaluar después del plan inicial:
CREATE INDEX idx_pagos_estado_fecha_afiliado ON pagos (estado, fecha, afiliado_id);
CREATE INDEX idx_afiliados_pais ON afiliados (pais);
ANALYZE afiliados; ANALYZE pagos;
```

Registrar tiempos reales antes/después y calcular mejora porcentual cuando tenga sentido: (tiempo_antes - tiempo_despues) / tiempo_antes × 100. Repetir varias mediciones si hay ruido por caché; comparar en igualdad de condiciones. Evidencia B3_explain_despues_indice.png y tabla de comparación.

B4 · Antipatrones SQL, 8 puntos

Explicar por qué SELECT * recupera columnas no utilizadas y por qué EXTRACT(YEAR FROM fecha) puede impedir el uso eficiente de un índice normal en fecha. Comparar una consulta con EXTRACT contra otra equivalente con rango desde el 1 de enero de 2024 hasta antes del 1 de enero de 2025. Demostrar igualdad de COUNT y comparar planes.

```
-- Filtro que se estudiará como antipatrón:
EXTRACT(YEAR FROM fecha) = 2024
```

```
-- Alternativa con rango equivalente:
fecha >= DATE '2024-01-01' AND fecha < DATE '2025-01-01'
```

Tomar B4_equivalencia_conteos.png y B4_planes_comparados.png. Evitar afirmar que toda consulta SELECT * es siempre lenta; explicar el contexto y la comparación observada.

9. Matriz de evidencias: qué capturar y cuándo

Código	Captura recomendada	Momento
B1	B1_conteo_inicial_postgresql.png	Consulta y resultado de 6363 / 317007.
B2	B2_explain_antes_indice.png	Antes de crear índices nuevos.
B3	B3_explain_despues_indice.png	Después de índice y ANALYZE, mismo SQL.
B4	B4_equivalencia_conteos.png	Ambas versiones y COUNT iguales.
B4	B4_planes_comparados.png	EXPLAIN de ambas versiones.
A1	A1_filtro_mongodb.png	Consulta y resultado real.
A2	A2_agrupacion_paises_mongodb.png	Pipeline y agregación ordenada.
A3	A3_antes_indice.png	Antes de createIndex.
A3	A3_despues_indice.png	Después de createIndex, mismo filtro.
A4	A4_sesion_redis.png	GET sesión, pais y carrito.
A4	A4_contador_redis.png	Cinco incrementos y valor resultante.
A4	A4_ttl_redis.png	EXPIRE y TTL visible antes de 120 segundos.

Buenas prácticas de capturas

• Capturar el comando y el resultado en la misma imagen siempre que sea legible.

• En Windows usar Win + Shift + S. Guardar PNG en evidencias/ivan/postgresql, evidencias/wilberth/mongodb o evidencias/wilberth/redis.

• La terminal, Mongo Shell o Redis CLI son evidencias válidas. Evitar capturas cortadas o que oculten cifras importantes.

• Acompañar cada imagen con número, descripción, qué demuestra y, si aplica, resumen de métricas.

• No volver a cargar la base para “mejorar” una captura, ni borrar índices/ejecutar comandos destructivos sin planificarlo.

10. C1 - Tabla de equivalencias (8 puntos)

Operación	PostgreSQL	MongoDB	Redis
Filtrar	SELECT / WHERE	find / countDocuments	GET por clave; sin filtro relacional equivalente
Insertar	INSERT	insertOne	SET
Actualizar	UPDATE	updateOne	SET / HSET / INCR según dato
Eliminar	DELETE	deleteOne	DEL
Agrupar o contar	GROUP BY / COUNT	aggregate / $group	INCR y contadores por clave
Plan de ejecución	EXPLAIN (ANALYZE, BUFFERS)	explain("executionStats")	No hay EXPLAIN análogo para GET/SET

La tabla es una guía conceptual: Redis no ofrece todas las operaciones relacionales o de agregación con semántica idéntica. Explicar diferencias y limitaciones, no forzar equivalencias falsas.

11. C2, C3 y el informe final

C2 · Reflexión de 8–12 líneas, 8 puntos

Responder con observaciones verdaderas: ¿qué mostró el plan de MongoDB antes y después del índice? ¿Qué operadores y costos reveló PostgreSQL? ¿Cuál es el costo de mantener índices durante INSERT/UPDATE/DELETE? ¿Cuándo elegir PostgreSQL por integridad y transacciones ACID, MongoDB por flexibilidad documental y Redis por sesiones/contadores? Relacionar ACID/BASE sin convertirlo en una etiqueta absoluta que ignore capacidades transaccionales reales.

C3 · Portada, orden y evidencia, 5 puntos

La portada debe incorporar UTN, curso, prueba corta, pareja, ambos carnés, caso 9, PAIS CO, EDAD 50, ANIO 2024, MONTO 150, USUARIO u241 y código 316164E0. Agregar introducción breve, apartados A, B, C, capturas numeradas, comentarios comparativos, conclusiones y, si corresponde, referencias a la documentación del curso.

12. Video demostrativo - máximo 10 minutos

Tiempo orientativo	Contenido	Responsable
0:00–0:45	Presentación, caso, ficha y tecnologías	Ambos
0:45–3:10	MongoDB A1/A2/A3; mostrar explain previo/posterior	Wilberth
3:10–4:20	Redis A4: sesión, contador y TTL	Wilberth
4:20–7:45	PostgreSQL B1/B2/B3/B4, consulta y EXPLAIN	Iván
7:45–9:30	Comparaciones C1/C2, conclusiones y cierre	Ambos

Se trata de una propuesta de reparto, no de una cronología exigida por el profesor. Ensayen antes para que ambas voces se escuchen, las letras en pantalla sean legibles y el video no sobrepase 10 minutos.

13. Control final antes de entregar

- Confirmar que cada consulta se ejecuta en el motor correcto con los datos personalizados.
- Verificar A3, B2 y B3 con capturas de planes antes/después en secuencia válida.
- Confirmar el COUNT equivalente de B4 y las observaciones de rendimiento.
- Revisar que el informe incluya todos los parámetros y el código de integridad.
- Confirmar que ambos participan en el video y que hay consulta en los tres motores.
- Comprobar que el PDF final abre bien y las capturas son legibles.
- Hacer merge revisado de ambas ramas hacia main; verificar repo y archivos finales.

14. Cómo utilizar esta guía con Claude o Codex

Wilberth puede proporcionar este PDF a Claude junto con acceso/copia del repositorio; el asistente deberá analizar el contenido efectivo de los archivos antes de ayudar. Si la herramienta no puede acceder a GitHub, adjuntar el README, ficha y archivos necesarios. No es obligatorio entregar el enunciado oficial si la guía contiene el contexto de trabajo, pero se recomienda compartirlo también para validar la rúbrica literal y resolver posibles ambigüedades.

Prompt inicial para Wilberth (copiar y pegar)

```
Estoy desarrollando la Prueba Corta 1 de Base de Datos Avanzadas
en pareja con Ivan. Lee primero el PDF adjunto y el README
del repositorio PruebaCorta1-Gimnasio. Mi responsabilidad es
la Parte A: MongoDB A1-A3 y Redis A4.

Trabajare en la rama wilberth-nosql. Mi dataset es el del
caso 9, carnes 208820443 y 208830343. No inventes resultados.
Guiame paso a paso, un comando a la vez, explicando objetivo,
resultado esperado y cuando tomar capturas. Antes de un indice
o de modificar o borrar datos, avisame y verifica que ya
registre la evidencia inicial. No alteres la Parte B de Ivan.
Al finalizar, ayudame a crear los archivos de consulta y una
explicacion sencilla para el informe y el video.
```

Prompt corto para Iván al iniciar la Parte B

```
Continuamos la Prueba Corta 1, caso 9 Gimnasio. Estoy en
la rama ivan-postgresql. Ya cargue PostgreSQL: 6363 afiliados
y 317007 pagos. Guiame por B1-B4, una actividad a la vez.
Antes de crear indices necesito guardar EXPLAIN inicial.
Indica siempre momento, nombre y ruta de las capturas,
y ayudame a interpretar mis resultados reales.
```

15. Qué falta hacer en orden

Fase	Próxima acción	Estado
0. Dataset	Generación, PostgreSQL, GitHub y ramas	Confirmado
1. Wilberth	Invitación GitHub y carga MongoDB/Redis	Pendiente
2. Desarrollo paralelo	Iván B2–B4; Wilberth A1–A4	Pendiente
3. Evidencias e informe	C1–C3 y revisión cruzada	Pendiente
4. Validación	Comandos, índices, capturas y video	Pendiente
5. Entrega	PDF final, video y archivos requeridos	Pendiente

Referencia de origen: enunciado oficial de la prueba compartido previamente, catálogo de casos, scripts del generador y archivos personalizados ficha.txt, postgres.sql, afiliados.json, redis.txt y cargar.txt. Las consultas aquí mostradas son ejemplos operativos que deben comprobarse en los contenedores reales.